

1. End-to-End Model

- **What it means:** Instead of splitting your pipeline into isolated phases (LightGBM predicting probabilities $\rightarrow$ separate calibration layer $\rightarrow$ separate shrinkage step), an end-to-end model unifies the data pipeline into a single, continuous differentiable network.
- **How to implement it:** Train a deep neural network (e.g., a PyTorch MLP or Transformer) directly optimized with a custom loss function (such as a calibrated focal loss or direct EV-maximizing utility loss). The model learns feature representations, calibration, and noise shrinkage simultaneously within its internal parameters.

2. Use More Features

- **What it means:** Moving beyond basic 5- and 10-game goal differentials and rest days to capture deeper, granular tactical and market signals.
- **How to implement it:**
 - **Advanced On-Pitch Metrics:** Expected Goals (xG), Expected Assists (xA), shot volume, non-penalty xG, deep completions, and defensive press intensity (PPDA).
 - **Player-Level Availability:** Lineup valuation, key player injuries, squad depth indices, and international duty fatigue.
 - **Dynamic Market Signals:** Opening odds vs. closing odds trends, market liquidity, and line movement velocity across multiple bookmakers.

3. Use Tabular Foundation Models

- **What it means:** Replacing or augmenting traditional tree-based models (like LightGBM) with modern deep learning backbones pre-trained or specifically designed for complex tabular data relationships.
- **How to implement it:**
 - **TabNet:** Uses sequential attention to select features at each decision step, mimicking tree structures while retaining neural network flexibility.
 - **FT-Transformer (Feature Tokenizer Transformer):** Converts continuous and categorical tabular features into embeddings processed by self-attention layers to capture subtle feature interactions.
 - **CARDS / TabPFN:** Zero-shot or fine-tuned foundation models built specifically to classify tabular datasets without heavy manual feature tuning.

4. Learn Strategy (Reinforcement Learning / End-to-End Execution)

- **What it means:** Instead of predicting probabilities and applying fixed heuristic rules (like $\$0.25 \times \text{Quarter-Kelly}$ and static thresholds), you train an agent to directly learn the optimal betting policy.
- **How to implement it:**
 - **Reinforcement Learning (RL):** Formulate match betting as a Markov Decision Process (MDP). An RL agent (e.g., PPO or DDPG) observes team state features and market odds (state), and outputs exact continuous stake amounts (action) to maximize risk-adjusted bankroll growth (reward).
 - **Direct Utility Maximization:** Replace classification loss with a custom loss function that optimizes log-bankroll growth directly during gradient descent.

5. 2-Stage Model

- **What it means:** Decoupling the pipeline into two specialized, modular machine learning stages instead of forcing one model to do everything.
- **How to implement it:**
 - **Stage 1 (Match Engine):** Train a baseline model strictly to predict true underlying match physical metrics (e.g., predicting expected goals, shot counts, or raw win probabilities) completely independent of betting market odds.
 - **Stage 2 (Market & Edge Model):** Take the predicted outputs from Stage 1 alongside live odds dynamics (PSH, MaxH, AvgH) and train a secondary meta-model (or regression engine) specifically tasked with predicting odds mispricing, market reaction, or optimal stake execution.